

Bugs

- name origin
 - Thomas Edison 1878: "little faults and difficulties" called "Bugs"
 - Admiral Grace Hopper 1940
 - moth stuck in relay
 - *"first actual bug being found"*
 - examples in [bugs](#)
-

Types of bugs

Heisenbugs

- bugs that disappear when observed
 - because `print` changes
 - timing
 - scheduling
 - optimization
 - cache behavior
 - ...
-

Race conditions

- applicable to [multi processing](#)
 - two [threads](#) increment the same variable
 - adding `print` can change the timing enough that the race-condition disappears
-

Async scheduling

- issue depends on **when** event loop switches tasks
 - adding `print` changes scheduling
-

State changing between prints

- applicable to [multi processing](#)
- some other [thread](#) modifies the object between `print` statements
- `print` does not tell you
 - where a variable came from
 - who modified it
 - whether it is referenced by some other object

```
print(obj)
#modification by another thread
print(obj)
```

Deadlocks

- two [threads](#) hold one lock and wait for the other
 - program hangs
-

Mutable object aliasing

```
a = []
b = a
b.append(1)
print(a)    # [1]
```

- `print()` is correct
 - real bug: `a` and `b` refer to the same object
-

Swallowed exceptions

```
try:
    some_func()
except Exception:
    pass
```

- better

```
try:
    some_func()
except Exception:
    import traceback
    traceback.print_exc()
```

Infinite loop

- script will just keep going
- debugger allows to see when things go wrong

```
#float precision issue
x = 0.0
while x != 1.0:
    x += 0.1
```

Memory corruption

- Segmentation fault
- Python protects against those
- can occur when calling into native code (C/C++)